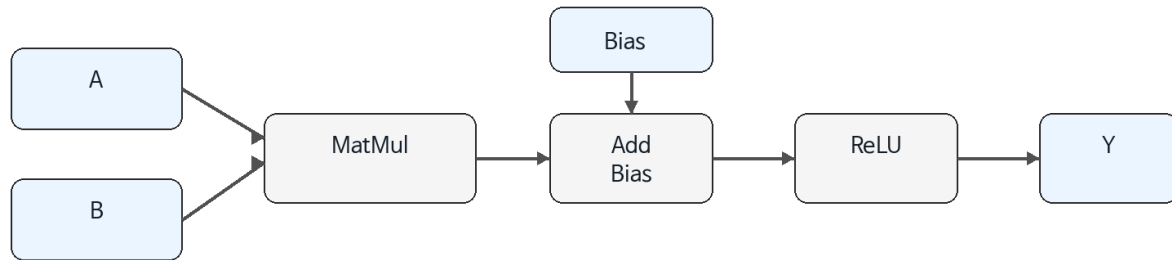


MLIR NPU Compiler Lecture 10

Rewrite Pattern

MatMul + Add + ReLU Fusion Pattern Design

MatMul + Add + ReLU Fusion: Before / After



After rewrite:

```
npu_graph.matmul_epilogue {bias=true, activation="relu"}
```

One target-aware op preserves shape/type contract and exposes epilogue scheduling opportunity.

- 강의 위치: 9 강 Pass Infrastructure 에서 pass 를 어떻게 등록하고 실행하는지 배웠다면, 10 강은 pass 내부에서 실제 IR 변환을 수행하는 가장 작은 단위인 rewrite pattern 을 다룹니다.
- 핵심 산출물: MatMul + Bias + ReLU 를 `npu_graph.matmul_epilogue` 로 fusion 하는 pattern 설계표, C++ skeleton, FileCheck positive/negative test.
- NPU 관점: fusion 은 단순 op 개수 축소가 아니라 epilogue kernel contract, accumulator/requant policy, SRAM lifetime, layout propagation 을 함께 증명하는 과정입니다.

1. Learning Objectives

- ``RewritePattern``, ``OpRewritePattern``, ``PatternRewriter``, ``RewritePatternSet`` 의 역할을 설명한다.
- ``matchAndRewrite`` 에서 match phase 와 rewrite phase 를 분리해 side-effect-free matching 을 수행한다.
- greedy rewrite driver 의 worklist/fixpoint 동작과 pattern benefit 의 의미를 이해한다.
- MatMul + Add + ReLU fusion pattern 의 semantic, use-def, shape, dtype, layout, target legality check 를 설계한다.
- positive/negative FileCheck test 와 ``notifyMatchFailure`` 기반 debug 흐름을 작성한다.

2. Rewrite Pattern 이란 무엇인가?

MLIR 의 pattern rewriting framework 는 pattern definition 과 pattern application 으로 나눌 수 있습니다. Pattern definition 은 어떤 root operation 을 대상으로 어떤 조건을 만족할 때 어떤 IR 로 바꿀지를 설명합니다. Pattern application 은 driver 가 pattern set 을 받아 worklist 를 돌며 benefit/cost model 에 따라 변환을 적용합니다.

구성 요소	역할	NPU compiler 예시
Root operation	pattern 이 match 를 시작하는 op	ReLU-like linalg.generic 또는 npu_graph.relu
Match predicates	shape/type/use/attribute/producer 조건 확인	bias broadcast 가능, MatMul 결과 one-use
Rewrite action	새 op 생성, 기존 op 대체/삭제	npu_graph.matmul_epilogue 생성
Benefit	동시에 match 되는 pattern 간 우선순위 신호	MatMul+Bias+ReLU > MatMul+Bias
Debug label	pattern filtering/debugging 단위	NPUFusion, EpilogueFusion

3. PatternRewriter 의 핵심 규칙

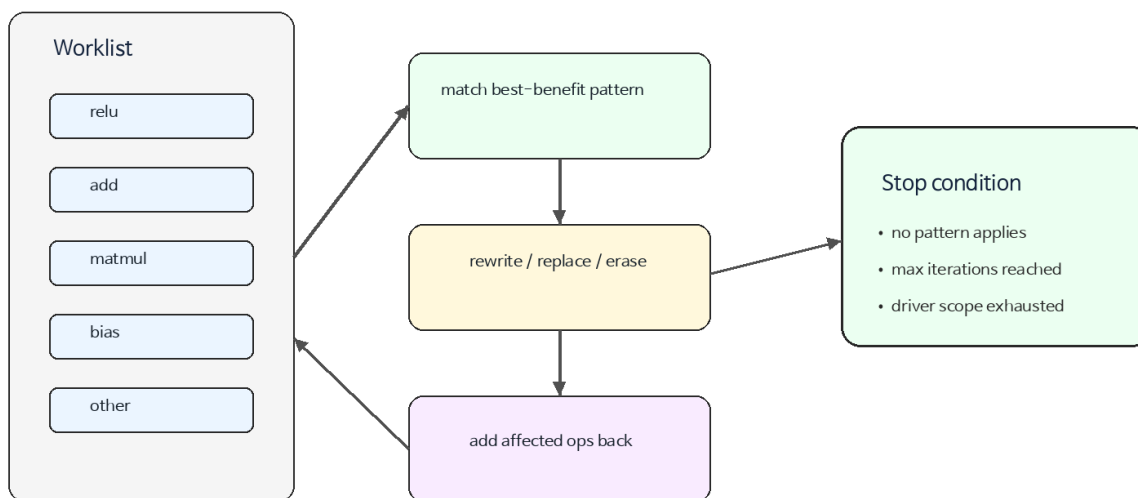
- 모든 IR mutation 은 `PatternRewriter` 를 통해 수행합니다. 직접 `op->erase()` 같은 mutation 을 하면 driver 의 worklist/state 가 깨질 수 있습니다.
- `matchAndRewrite` 는 성공 시 IR 을 실제로 변경해야 하고, 실패 시에는 IR 을 변경하지 않아야 합니다.
- root operation 은 update-in-place, replace, erase 중 하나로 처리되어야 합니다.
- mutation 전에는 shape, type, use-def, attribute, target capability 를 모두 확인합니다.
- 실패 조건은 `rewriter.notifyMatchFailure(op, "reason")` 로 남겨 debug log 와 test reduction 에 활용합니다.

```
LogicalResult matchAndRewrite(MyOp op, PatternRewriter &rewriter) const override {
    if (!predicateA(op))
        return rewriter.notifyMatchFailure(op, "predicate A failed");
    if (!predicateB(op))
        return rewriter.notifyMatchFailure(op, "predicate B failed");

    // Rewrite starts only after all required checks are proven.
    auto newOp = rewriter.create<NewOp>(op.getLoc(), op.getResultTypes(), op.getOperands());
    rewriter.replaceOp(op, newOp.getResults());
    return success();
}
```

4. Greedy Rewrite Driver 이해

Greedy Rewrite Driver: Worklist and Fixpoint



Greedy driver 는 초기 worklist 에 들어온 operation 에 대해 지역적으로 가장 높은 benefit 의 pattern 을 적용합니다. 변환 된 op 와 새로 생성된 op 는 다시 worklist 에 들어갈 수 있고, 더 이상 적용 가능한 pattern 이 없거나 max iteration/rewrite limit 에 도달하면 멈춥니다. 이 구조 때문에 pattern 은 수렴해야 하며, cyclic rewrite 를 만들면 안 됩니다.

항목	권장값/원칙	이유
Traversal	top-down 또는 bottom-up 을 pass 특성에 맞게 선택	큰 graph pattern 은 producer/consumer 순서에 민감
maxIterations	NPU fusion pass 에서는 작은 상한부터 시작	cyclic rewrite 를 빠르게 발견
pattern benefit	더 구체적이고 안전한 pattern 에 높은 benefit	MatMul+Bias+ReLU 가 MatMul+Bias 보다 먼저 적용
scope	func.func 또는 npu_graph.cluster 단위	region 밖 op 를 불필요하게 재방문하지 않음

5. Canonicalization Pattern vs Optimization Pattern

Canonicalization 은 후속 분석/최적화가 쉬운 표준 형태로 IR 을 정리하는 best-effort 변환입니다. 반면 MatMul+Bias+ReLU fusion 은 NPU target capability 를 반영하는 optimization pattern 입니다. 따라서 canonicalization pass 가 없어도 pipeline correctness 가 유지되어야 하며, target-specific fusion 은 별도 pass 에서 명확한 legality check 를 수행하는 편이 좋습니다.

구분	Canonicalization	NPU Target Fusion
목표	IR 단순화, 중복 제거, 표현 안정화	target epilogue kernel 로 성능/메모리 최적화
전제	일반적이고 저렴한 match	hardware feature, dtype, layout, SRAM budget
실패 영향	보통 성능/분석 편의성 저하	잘못되면 numerical/correctness bug 가능
예시	x + 0 제거, constant folding	MatMul + Bias + ReLU -> npu_graph.matmul_epilogue

6. Case Study: MatMul + Add + ReLU Fusion

NPU Fusion Contract: What the Pattern Must Prove

Semantic	Shape/Layout	DType/Quant	Use/Memory	Target
MatMul + bias + relu exactly preserves math	Output rank/static dims, bias broadcast legal	Accumulator, requant, saturation policy legal	MatMul/add/relu producers not shared unsafely	Epilogue supported by target kernel
No hidden side effects or ordering dependency	Layout transform cost acceptable	INT8/INT4 scale/zero-point compatible	Fusion reduces intermediate tensor lifetime	SRAM footprint within tile budget

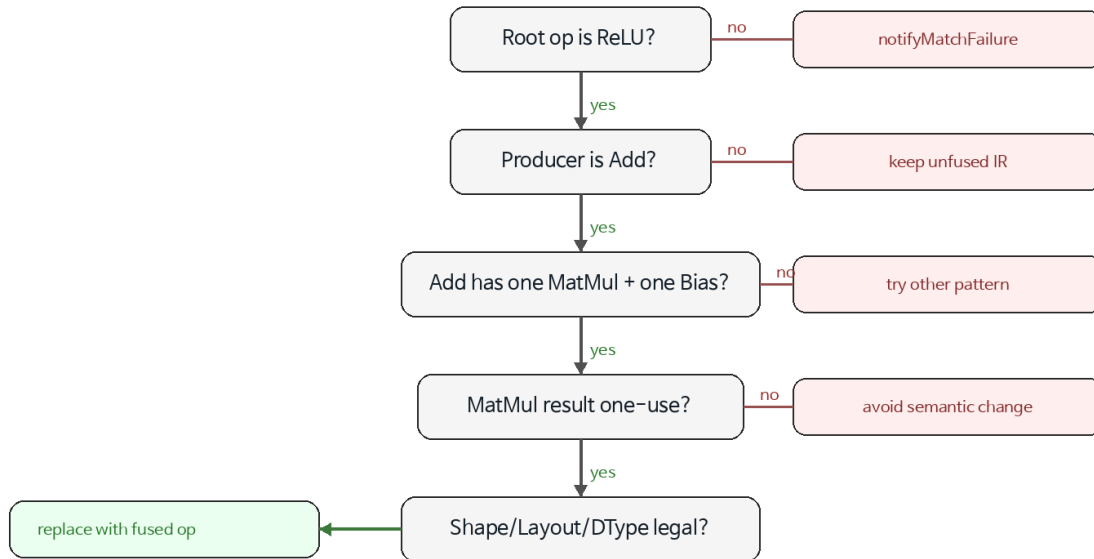
Outcome: use notifyMatchFailure for every failed proof; keep unfused IR if a contract cannot be proven.

Fusion pattern 은 root 를 ReLU 로 두는 것이 일반적으로 유리합니다. ReLU 에서 producer 를 따라 AddBias 와 MatMul 을 역방향으로 확인하면 output consumer 기준으로 pattern 전체를 담을 수 있고, replacement op 의 결과가 기존 ReLU 결과를 그대로 대체할 수 있습니다.

단계	검사	실패 시 이유
1	root op 가 ReLU-like 인가?	activation semantics mismatch
2	ReLU input producer 가 AddBias-like 인가?	producer is not add/bias
3	Add operand 중 하나가 MatMul result, 다른 하나가 bias 인가?	cannot classify add inputs
4	MatMul/Add result 가 one-use 인가?	shared producer result
5	shape/broadcast/layout/dtype 이 target epilogue 와 호환되는가?	NPU fusion contract failed
6	replacement result type 이 기존 ReLU result type 과 동일한가?	type mismatch

7. Matcher Decision Tree

Matcher Decision Tree: Fuse or Reject



좋은 matcher 는 rejection path 가 명확합니다. 특히 NPU compiler 에서는 왜 fusion 되지 않았는지가 중요합니다. 예를 들어 shared producer, unsupported bias broadcast, unsupported dtype, layout mismatch 는 모두 서로 다른 failure reason 으로 남겨야 tuning/debug 가 가능합니다.

8. C++ Pattern Skeleton

```
// Lecture 10 - pseudo C++ pattern skeleton.
// Purpose: show match discipline and PatternRewriter usage, not a drop-in build file.

#include "mlir/IR/PatternMatch.h"
#include "mlir/Transforms/GreedyPatternRewriteDriver.h"

using namespace mlir;

namespace {

struct FuseMatMulBiasReluPattern : public OpRewritePattern<linalg::GenericOp> {
    using OpRewritePattern<linalg::GenericOp>::OpRewritePattern;

    void initialize() {
        setDebugName("FuseMatMulBiasReluPattern");
        addDebugLabels("NPUFusion", "EpilogueFusion");
    }

    LogicalResult matchAndRewrite(linalg::GenericOp relu,
                                  PatternRewriter &rewriter) const override {
        // 1) Match root as ReLU-like generic op.
        if (!isReluGeneric(relu))
            return rewriter.notifyMatchFailure(relu, "root is not ReLU generic");

        // 2) Match producer Add.
        auto add = getSingleProducerOfType<linalg::GenericOp>(relu.getDpsInputOperand(0));
        if (!add || !isAddBiasGeneric(add))
```

```

        return rewriter.notifyMatchFailure(relu, "producer is not AddBias");

// 3) Match MatMul input and bias operand.
Value matmulResult;
Value bias;
if (failed(classifyAddInputs(add, matmulResult, bias)))
    return rewriter.notifyMatchFailure(add, "cannot classify matmul and bias");

auto matmul = getSingleProducerOfType<linalg::GenericOp>(matmulResult);
if (!matmul || !isMatMulGeneric(matmul))
    return rewriter.notifyMatchFailure(add, "input is not matmul");

// 4) Prove safety and target legality.
if (!matmul->hasOneUse() || !add->hasOneUse())
    return rewriter.notifyMatchFailure(relu, "producer result is shared");
if (failed(checkShapeDTypeLayoutForTarget(matmul, add, relu, bias)))
    return rewriter.notifyMatchFailure(relu, "NPU fusion contract failed");

// 5) Rewrite only after all checks have succeeded.
Location loc = rewriter.getFusedLoc({matmul.getLoc(), add.getLoc(), relu.getLoc()},
                                     "npn.matmul_bias_relu");
Value lhs = matmul.getDpsInputOperand(0)->get();
Value rhs = matmul.getDpsInputOperand(1)->get();
Type resultType = relu.getResult(0).getType();

auto fused = rewriter.create<npu_graph::MatmulEpilogueOp>(
    loc, resultType, lhs, rhs, bias,
    /*activation=*/rewriter.getStringAttr("relu"));

rewriter.replaceOp(relu, fused.getResults());
rewriter.eraseOp(add);
rewriter.eraseOp(matmul);
return success();
}
};

} // namespace

// ... see example file for the full skeleton ...

```

9. Pattern Set 과 Pass 연결

```

void runOnOperation() override {
    RewritePatternSet patterns(&getContext());
    patterns.add<FuseMatMulBiasReluPattern>(&getContext());

    GreedyRewriteConfig config;
    config.useTopDownTraversal = true;
    config.maxIterations = 10;

    if (failed(applyPatternsGreedily(getOperation(), std::move(patterns), config)))
        signalPassFailure();
}

```

- NPU fusion pass 는 `func.func` 단위 또는 `npu\_graph.cluster` 단위로 실행하는 것이 일반적입니다.
- Pattern set 에는 일반 pattern 과 target-specific pattern 을 섞을 수 있지만, benefit 과 legality 가 명확해야 합니다.
- Pass 가 `npu\_graph.matmul\_epilogue` 를 생성한다면 dependent dialect 에 npu_graph dialect 를 반드시 등록해야 합니다.

10. FileCheck Regression Test

```
// RUN: mlir-opt %s --npufuse-epilogue | FileCheck %s
// CHECK-LABEL: func.func @matmul_bias_relu
// CHECK: "npugraph.matmul_epilogue"
// CHECK-SAME: activation = "relu"
// CHECK-NOT: linalg.generic
```

Positive test 는 fused op 가 생겼는지뿐 아니라 기존 unfused op 가 사라졌는지 확인합니다. Negative test 는 shared MatMul result, unsupported bias layout, unsupported dtype 같은 케이스가 fusion 되지 않음을 확인해야 합니다.

11. NPU Fusion Contract 상세

Contract	확인 내용	Reject 예시
Semantic	MatMul, AddBias, ReLU 의 수학적 의미가 fused epilogue 와 동일해야 함	ReLU 가 $\max(x, 0)$ 가 아니라 clamp6 이면 다른 activation
Use-def	fusion 으로 제거되는 intermediate result 가 외부에서 사용되지 않아야 함	MatMul result 가 residual branch 에도 연결됨
Shape	bias broadcast, batch dimension, output rank 가 target 에서 지원되어야 함	bias 가 [M,N] full tensor 라 epilogue scalar/vector bias 불가
DType/Quant	accumulator 와 output dtype, scale, zero-point, saturation 이 표현 가능해야 함	per-token quant 인데 kernel 은 per-channel 만 지원
Layout	lhs/rhs/out layout 과 transpose attribute 가 target kernel 과 일치해야 함	rhs packed layout 이 아직 생성되지 않음
Memory	fusion 후 SRAM tile footprint 가 감소하거나 적어도 증가하지 않아야 함	bias prefetch 가 SRAM budget 초과
Runtime ABI	fused op 가 command buffer descriptor 로 emit 가능해야 함	activation enum 이 ABI 에 없음

12. Common Mistakes

- Match 중간에 op 를 먼저 생성하거나 기존 op 를 수정한 뒤 실패하는 pattern.
- one-use 조건을 확인하지 않고 producer 를 erase 하는 pattern.
- broadcast semantics 가 다른 bias add 를 epilogue bias 로 잘못 취급하는 pattern.
- PatternBenefit 만 높이고 legality check 가 약해 다른 안전한 pattern 을 밀어내는 경우.
- canonicalization 에 target-specific fusion 을 넣어 pipeline correctness 와 target 옵션을 섞는 경우.
- positive test 만 있고 negative test 가 없어 unsupported case 가 조용히 fusion 되는 경우.

13. 실습

- Exercise A: `01\_linalg\_matmul\_add\_relu\_before.mlir` 에서 ReLU root, Add producer, MatMul producer, bias operand 를 색인한다.
- Exercise B: `03\_matmul\_add\_relu\_fusion\_pattern.cpp` 의 helper function 목록과 failure reason 을 채운다.
- Exercise C: `08\_fusion\_contract\_checklist.md` 를 기준으로 fusion 허용/거절 case 5 개를 설계한다.
- Exercise D: `06\_filecheck\_fusion\_test.mlir` 와 `07\_negative\_no\_fusion\_test.mlir` 의 CHECK line 을 보장한다.

14. 과제

과제	요구사항	제출물
Fusion pattern spec	MatMul+Bias+ReLU pattern 의 root, predicates, replacement, legality 작성	1-page design spec
C++ skeleton completion	notifyMatchFailure reason 8 개 이상, target legality helper 분리	pattern .cpp
Regression matrix	positive 3 개, negative 5 개 test 설계	FileCheck .mlir
NPU ABI impact	fused op descriptor field 와 runtime launch parameter 정리	descriptor table

15. References

- MLIR Pattern Rewriter: <https://mlir.llvm.org/docs/PatternRewriter/>
- MLIR Canonicalization: <https://mlir.llvm.org/docs/Canonicalization/>
- MLIR Dialect Conversion: <https://mlir.llvm.org/docs/DialectConversion/>
- MLIR Quickstart Rewrites: <https://mlir.llvm.org/docs/Tutorials/QuickstartRewrites/>
- MLIR GreedyPatternRewriteDriver API: https://mlir.llvm.org/doxygen/GreedyPatternRewriteDriver_8h.html